ETEC TABOÃO DA SERRA

Técnico em Informática para Internet

Interface Web II

GUSTAVO OLIVEIRA DOS SANTOS

PAULO HENRIQUE DE LIMA SILVA

ATIVIDADE: NOVA FUNCIONALIDADE COM IA

Painel do Esporte: gráfico de gols por jogador com Chart.js

Documento de evidências

Professor: Clayton de Almeida Souza

Turma: 2°C

TABOÃO DA SERRA

2026

SUMÁRIO

1 Identificação....3
1.1 Trabalho em dupla (piloto e copiloto)....3
2 Etapa 0: como a página carrega o arquivo....4
2.1 Escolha da trilha e planejamento....4
3 Pedidos à IA e evidências....6
3.1 Quadro de evidência 1: criar o gráfico....6
3.2 Quadro de evidência 2: depurar o erro do canvas....8
3.3 Quadro de evidência 3: plano B, cores e tema....11
4 Código final comentado....15
4.1 index.html (seção nova)....15
4.2 style.css (bloco novo)....15
4.3 script.js (BLOCO 7 completo)....15
5 Funcionalidade funcionando....18
6 Registro de testes....20
7 Justificativa da biblioteca (Chart.js)....22
8 Fechamento e reflexão....23
8.1 O que aprendemos que não sabíamos antes de usar a IA?....23
8.2 Em que ponto a IA mais atrapalhou?....23
8.3 Onde a IA errou (resumo)....23
8.4 Roteiro para a apresentação (3 minutos)....23
9 Checklist de entrega....24
Referências....25

1 Identificação

Campo	Resposta
Integrantes	Gustavo Oliveira dos Santos e Paulo Henrique de Lima Silva
Turma	2°C, Técnico em Informática para Internet
Nosso projeto	☐ Rateio do Churrascão ☒ Painel do Esporte
Nossa trilha	☐ 1 Dados novos ☐ 2 API ☒ 3 Biblioteca (Chart.js 4.5.1)
Data de início	04/11/2026
Entrega	04/12/2026

1.1 Trabalho em dupla (piloto e copiloto)

Seguimos a regra da atividade: quem está no teclado é o piloto, o outro confere cada linha e anota no Word. Trocamos de papel a cada etapa:

Etapa	Piloto (digita)	Copiloto (confere)
1 e 2: escolha e perguntas da Etapa 0	Gustavo	Paulo
3: prompts e código da IA	Paulo	Gustavo
4: testes, correções e comentários	Gustavo	Paulo
5 e 6: Word, PDF, ZIP e ensaio da fala	Paulo	Gustavo

2 Etapa 0: como a página carrega o arquivo

Antes de pedir qualquer coisa à IA, abrimos o script.js original (BLOCO 6) e respondemos com nossas palavras. Os números de linha são do script.js original do professor.

Pergunta	Nossa resposta
1) Qual linha faz a tabela aparecer cheia assim que a página abre, sem ninguém clicar?	A linha 247: $("#btnJson").trigger("click");. O trigger "finge" um clique no botão Carregar JSON. Como esse botão tem um .on("click") que chama o $.getJSON, o arquivo é pedido sozinho quando a página abre, e o .done chama o renderizar() que preenche a tabela.
2) O que o .done faz? E o .fail?	O $.getJSON é assíncrono: ele faz o pedido e o código continua. O .done é a função que roda quando a resposta chega certa: guarda os dados em jogadores, chama renderizar() e mostra a mensagem verde. O .fail roda quando dá errado (arquivo não encontrado, JSON com erro, sem servidor) e mostra a mensagem vermelha "Falha ao carregar o JSON. Use o Live Server."
3) Por que, abrindo o index.html com duplo clique, os botões Carregar não funcionam?	Com duplo clique o endereço fica file:// e o navegador bloqueia o AJAX por segurança (uma página do disco não pode ler outros arquivos do disco por pedido). Aí o pedido cai no .fail. Pelo Live Server o endereço é http://127.0.0.1:5500 e o pedido funciona.
4) No XML, por que o código usa Number() ou === "true"?	Porque no XML tudo é texto. <gols>15</gols> vira a string "15", não o número 15. Sem o Number(), a soma de gols juntaria textos ("15"+"6" = "156") e a média daria errado. No Rateio, o pago vem como "true" (texto) e o === "true" transforma em boolean de verdade. No nosso projeto (Painel) usamos Number() em jogos e gols (linhas 226 e 227).

2.1 Escolha da trilha e planejamento

Escolhemos o **Painel do Esporte** e a **Trilha 3 (biblioteca)**, com o gráfico de gols por jogador que se atualiza sozinho. Pensamos primeiro na Trilha 2 (previsão do tempo), mas o gráfico pode ser testado sem depender de uma API externa e conversa direto com o que a tabela já calcula (gols e média).

Planejamento (3 linhas):

- **O que faz:** mostra um gráfico de barras com os gols (ou a média) de cada jogador, que muda junto com a tabela ao carregar, adicionar, remover, buscar, filtrar, ordenar e trocar o tema.
- **Arquivos e biblioteca:** Chart.js 4.5.1 pela CDN jsDelivr com cópia local (chart.umd.min.js); mexemos em index.html, style.css e script.js (BLOCO 7 novo). O jogadores.json continua o mesmo.
- **ids novos no HTML:** #graficoGols (canvas), #btnMetrica (gols/média), #tituloGrafico e #graficoAviso (mensagens do gráfico).

3 Pedidos à IA e evidências

Usamos o Claude (Anthropic) como assistente. Cada prompt tem as 4 partes pedidas (contexto, instrução, entrada e formato). Quando colamos código no prompt, ele aparece entre colchetes para o Word não ficar repetido; o código colado é o original do projeto.

3.1 Quadro de evidência 1: criar o gráfico

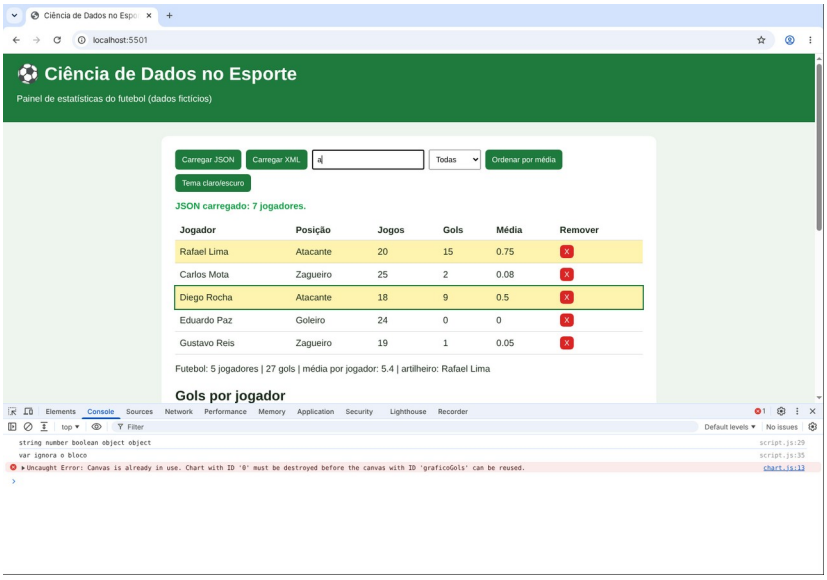
Item	Preenchido pela dupla
Prompt enviado à IA (inteiro)	CONTEXTO: Somos uma dupla do 2º ano do técnico em Informática para Internet da ETEC Taboão da Serra. Nosso projeto "Painel do Esporte" usa HTML, CSS, JavaScript e jQuery ($.getJSON, $.ajax com XML, .done/.fail, try/catch, eventos, cookies). Não usamos frameworks nem npm, abrimos tudo pelo Live Server. Os dados são fictícios. INSTRUÇÃO: Crie um gráfico de barras com os gols de cada jogador usando a biblioteca Chart.js, que se atualize sozinho quando a tabela mudar (carregar JSON ou XML, adicionar, remover, buscar e filtrar). ENTRADA: [colamos aqui o index.html inteiro, a função renderizar() do BLOCO 3, o BLOCO 6 do script.js e o arquivo jogadores.json, sem nenhuma alteração] FORMATO: Mostre só o que devemos ACRESCENTAR (HTML, CSS e JS), com comentário em cada linha, no estilo do nosso código. Antes, explique em 5 linhas o que vai mudar.
Explicação da IA (resumo das 5 linhas)	1) Adicionar o Chart.js pela CDN no <head>. 2) Criar uma seção com um <canvas> abaixo da tabela. 3) Criar a função desenharGrafico() no script.js. 4) Usar map() para tirar nomes e gols do array jogadores. 5) Chamar desenharGrafico() no fim do renderizar() para o gráfico se atualizar sozinho.

Código criado pela IA (sem mexer)	**HTML:**
	`<!-- Chart.js pela CDN (coloque no <head>, antes do script.js) -->`
	`<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>`
	
	`<!-- SEÇÃO NOVA: gráfico de gols (coloque depois da seção da tabela) -->`
	`<section class="grafico">`
	`  <h2>Gols por jogador</h2>  <!-- título da seção -->`
	`  <canvas id="graficoGols"></canvas>  <!-- o Chart.js desenha dentro deste canvas -->`
	`</section>`
	
	CSS:
	`/* Área do gráfico */`
	`.grafico { margin-top: 24px; }  /* espaço acima do gráfico */`
	`#graficoGols { max-height: 320px; }  /* limita a altura do gráfico */`

Item	Preenchido pela dupla

	JavaScript:
	`// ==========...` (BLOCO 7: GRÁFICO DE GOLS COM CHART.JS)
	`function desenharGrafico() {`
	`  const ctx = document.getElementById("graficoGols");` // pega o canvas pelo id
	`  new Chart(ctx, {` // cria o gráfico
	`  type: "bar",` // tipo: barras
	`  data: {`
	`  labels: jogadores.map(j => j.nome),` // nomes no eixo X
	`  datasets: [{`
	`  label: "Gols",` // legenda
	`  data: jogadores.map(j => j.gols),` // gols de cada jogador
	`  backgroundColor: "#1f7a3d"` // verde do projeto
	`  }]`
	`  },`
	`  options: {`
	`  scales: { y: { beginAtZero: true } }` // eixo Y começa no zero
	`  }`
	`  });`
	`}`
	
	`// No final da função renderizar(), acrescente:`
	`desenharGrafico();` // redesenha o gráfico

O que adaptamos e por quê	• Colamos do jeito que veio (versão "v1-ia") para testar antes de mudar. Ao digitar uma letra na busca apareceu erro vermelho no Console (ver print). Levamos o erro para o Prompt 2. • Percebemos que a IA usou jogadores.map(...): o gráfico mostrava sempre TODOS os jogadores, mesmo com filtro "Atacante". Na versão final passamos a lista filtrada (listaGrafico) que o renderizar() já monta. • A CDN veio sem versão (npm/chart.js). Se sair uma versão nova com mudanças, nosso código pode quebrar. Fixamos chart.js@4.5.1. • A IA usou arrow functions (j => j.nome). Funciona, mas o projeto inteiro usa function () {}. Trocamos para ficar no mesmo estilo e para conseguirmos explicar. • O max-height no canvas não segura bem o gráfico responsivo. Colocamos o canvas dentro de uma .grafico-caixa com altura fixa e maintainAspectRatio: false.
Trecho final, comentado por nós	Linhas [NOVO] que ligam o gráfico ao renderizar() (script.js final): `29:  let grafico = null;` // [NOVO] guarda o gráfico do Chart.js depois de criado (null = ainda não existe) `30:  let metrica = "gols";` // [NOVO] o que o gráfico mostra: "gols" ou "media" `70:  if (grafico !== null) { atualizarGrafico(grafico.listaAtual); }` // [NOVO] troca as cores do gráfico junto com o tema `101:  const listaGrafico = [];` // [NOVO] jogadores que passaram na busca/filtro (o gráfico mostra os mesmos da tabela) `122:  listaGrafico.push(j);` // [NOVO] guarda o jogador para o gráfico

Item	Preenchido pela dupla
	`139:  atualizarGrafico(listaGrafico);  // [NOVO] toda vez que a tabela muda, o gráfico muda junto`
A IA errou ou inventou algo? O que, e como percebemos?	Sim. O código cria um new Chart() TODA vez que o renderizar() roda. O Chart.js não deixa dois gráficos no mesmo canvas, então na primeira letra da busca apareceu: "Canvas is already in use. Chart with ID '0' must be destroyed...". Percebemos pelo Console (F12). Também notamos, comparando a tabela com o gráfico, que o filtro não era respeitado.
Print (Console F12 visível)	 Print 1: versão da IA sem mudanças. Ao digitar "a" na busca, a tabela filtra, mas o Console mostra o erro do canvas. 04/11/2026.

3.2 Quadro de evidência 2: depurar o erro do canvas

Item	Preenchido pela dupla
Prompt enviado à IA (inteiro)	CONTEXTO: Estamos adicionando o gráfico de gols com Chart.js no Painel do Esporte (jQuery, sem frameworks). Colamos o código que você mandou e abrimos pelo Live Server. MEU CÓDIGO: [colamos a função desenharGrafico() e o final do renderizar() onde ela é chamada] ERRO NO CONSOLE: Uncaught Error: Canvas is already in use. Chart with ID '0' must be destroyed before the canvas with ID 'graficoGols' can be reused. ESPERADO: o gráfico deveria mudar quando digitamos na busca, sem erro. O erro aparece logo na primeira letra digitada. Explique a causa antes de corrigir, linha por linha, como se estivéssemos no 2º ano.
Explicação	A IA explicou que o Chart.js "prende" o canvas ao gráfico criado.

Item	Preenchido pela dupla
da IA (resumo)	Como o renderizar() roda de novo a cada letra, o código tentava criar um segundo gráfico no mesmo canvas. Ela deu duas saídas: destruir o gráfico antigo antes de criar outro (destroy) ou criar uma vez só e depois trocar os dados com update().

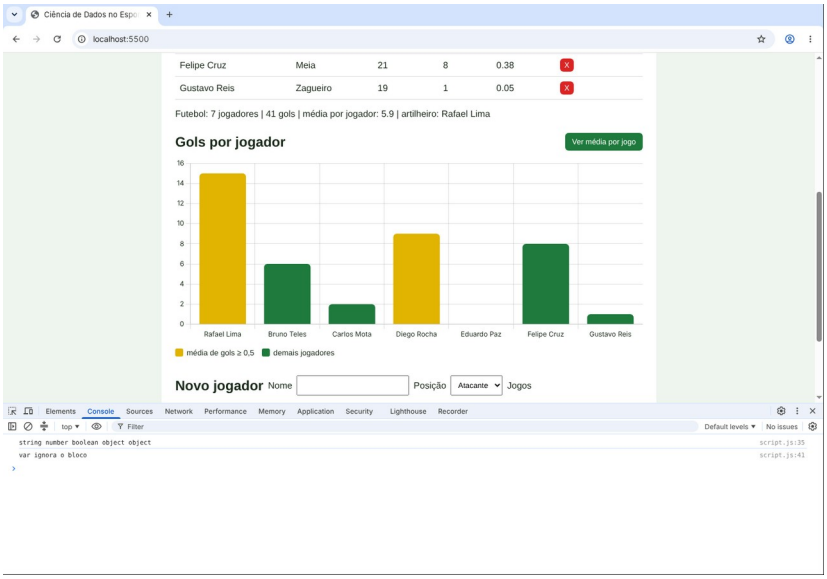
Código criado pela IA (sem mexer)	`// Variável fora da função para guardar o gráfico já criado`
	`let grafico = null;`
	
	`function desenharGrafico() {`
	`  const ctx = document.getElementById("graficoGols");`
	`  if (grafico) {  // se já existe um gráfico no canvas...`
	`  grafico.destroy();  // ...destrói antes de criar outro`
	`  }`
	`  grafico = new Chart(ctx, {  // guarda o novo gráfico na variável`
	`  type: "bar",`
	`  data: {`
	`  labels: jogadores.map(j => j.nome),`
	`  datasets: [{ label: "Gols", data: jogadores.map(j => j.gols), backgroundColor: "#1f7a3d" }]`
	`  },`
	`  options: { scales: { y: { beginAtZero: true } } }`
	`  });`
	`}`
	
	`// Outra opção (mais leve): criar uma vez e depois só trocar os dados`
	`// grafico.data.labels = novosNomes;`
	`// grafico.data.datasets[0].data = novosGols;`
	`// grafico.update();`

O que adaptamos e por quê	• Testamos as duas ideias. Com destroy() o erro some, mas o gráfico é apagado e desenhado do zero a cada letra digitada: pisca e perde a animação. Escolhemos a segunda opção (criar uma vez e usar update()), que a própria IA deixou só como comentário. • A variável grafico foi para o BLOCO 1, junto com as outras variáveis do projeto (ver Quadro 3, explicamos o motivo). • Mantivemos o if (grafico === null) para saber se é a primeira vez.

Trecho final, comentado por nós	`  if (grafico === null) {  // 1ª vez: cria o gráfico`
	`  grafico = new Chart(document.getElementById("graficoGols"), {`
	`  type: "bar",  // gráfico de barras`
	`  data: { labels: [], datasets: [{ label: "Gols", data: [], backgroundColor: [], borderRadius: 6 }] },`
	`  options: {`
	`  responsive: true,  // acompanha o tamanho da tela`
	`  maintainAspectRatio: false,  // usa a altura da .grafico-caixa do CSS`
	`  plugins: { legend: { display: false } },  // a legenda de cores já está no HTML`
	`  scales: {`
	`  x: { ticks: {}, grid: {} },`
	`  y: { beginAtZero: true, ticks: {}, grid: {} } // eixo Y começa no zero`
	`  }`

Item	Preenchido pela dupla

	`}`
	`});`
	`}`
	
	`// Daqui pra baixo roda SEMPRE: troca os dados do gráfico que já existe`
	`grafico.listaAtual = lista;  // guardo a lista para o aplicarTema() redesenhar com ela`
	`grafico.data.labels = nomes;`
	`grafico.data.datasets[0].data = valores;`
	`grafico.data.datasets[0].backgroundColor = cores;`
	`grafico.data.datasets[0].label = metrica === "gols" ? "Gols" : "Média de gols por jogo";`
	`// gols são inteiros (precision 0); média tem casas decimais (precision 2)`
	`grafico.options.scales.y.ticks.precision = metrica === "gols" ? 0 : 2;`
	`// cores das letras e da grade em cada eixo (forEach passa por "x" e "y")`
	`["x", "y"].forEach(function (eixo) {`
	`grafico.options.scales[eixo].ticks.color = corTexto;`
	`grafico.options.scales[eixo].grid.color = corGrade;`
	`});`
	`grafico.update();  // manda o Chart.js redesenhar com os dados novos`

A IA errou ou inventou algo?	Aqui a explicação estava certa. O problema é que a solução principal (destroy) funciona mas não é a melhor para um gráfico que muda a cada letra. A IA também manteve jogadores.map() mesmo depois de ver nosso renderizar() com busca e filtro, ou seja, o problema do filtro continuou e nós mesmos corrigimos.
Print (Console F12 visível)	 Print 2: versão final aberta pelo servidor na porta 5500. Gráfico com 7 barras, amarelo para média ≥ 0,5, Console sem erros. 04/11/2026.

3.3 Quadro de evidência 3: plano B, cores e tema

Item	Preenchido pela dupla
Prompt enviado à IA (inteiro)	CONTEXTO: Somos alunos do 2º ano (Informática para Internet, ETEC Taboão da Serra). Nosso gráfico Chart.js já funciona no Painel do Esporte. No index.html o jQuery tem um "plano B": se a CDN falhar, carrega a cópia local com window.jQuery || document.write(...). Testamos sem internet (F12 > Network > Offline) e apareceu "Chart is not defined" no Console e a mensagem "JSON carregado" sumiu. INSTRUÇÃO: (1) faça o mesmo plano B para o Chart.js, com versão fixa; (2) se mesmo assim a biblioteca não carregar, mostre um aviso na tela sem quebrar a tabela; (3) pinte de amarelo as barras dos jogadores com média >= LIMITE_DESTAQUE, igual à tabela; (4) faça o gráfico acompanhar o tema claro/escuro; (5) o gráfico deve mostrar só os jogadores que aparecem na tabela (busca e filtro). ENTRADA: [colamos o <head> do index.html, o BLOCO 1, o BLOCO 2 (aplicarTema), a função renderizar() e a função desenharGrafico() atual] FORMATO: Explique em até 5 linhas o que muda e mostre só o código novo, comentado, no estilo do nosso projeto (function, não arrow function).
Explicação da IA (resumo)	Usar o mesmo truque do jQuery (window.Chart || document.write) com a cópia local; testar typeof Chart antes de usar; criar um array de cores com map() comparando mediaGols(j) com LIMITE_DESTAQUE; trocar Chart.defaults.color conforme o tema; receber a lista filtrada como parâmetro.

Código criado pela IA (sem mexer)	**HTML (<head>):**
	`<!-- Chart.js com plano B local, igual ao jQuery -->`
	`<script src="https://cdn.jsdelivr.net/npm/chart.js@4.5.1/dist/chart.umd.min.js"></script>`
	`<script>window.Chart || document.write('<script src="chart.umd.min.js"><\/script>');</script>`
	
	JavaScript:
	`function desenharGrafico(lista) {`
	`  if (typeof Chart === "undefined") {  // a biblioteca não carregou`
	`  $("#graficoAviso").text("Gráfico indisponível: Chart.js não carregou.");`
	`  return;`
	`  }`
	`  // cor da barra: amarelo se a média for alta, verde se não`
	`  const cores = lista.map(j => mediaGols(j) >= LIMITE_DESTAQUE ? "#f5c400" : "#1f7a3d");`
	`  const corTexto = $("body").hasClass("escuro") ? "#e8f3e8" : "#1b2b1b";`
	`  Chart.defaults.color = corTexto;  // cor das letras do gráfico`
	`  if (grafico === null) {`
	`  grafico = new Chart(document.getElementById("graficoGols"), {`
	`  type: "bar",`
	`  data: { labels: [], datasets: [{ label: "Gols", data: [], backgroundColor: [] }] },`

Item	Preenchido pela dupla

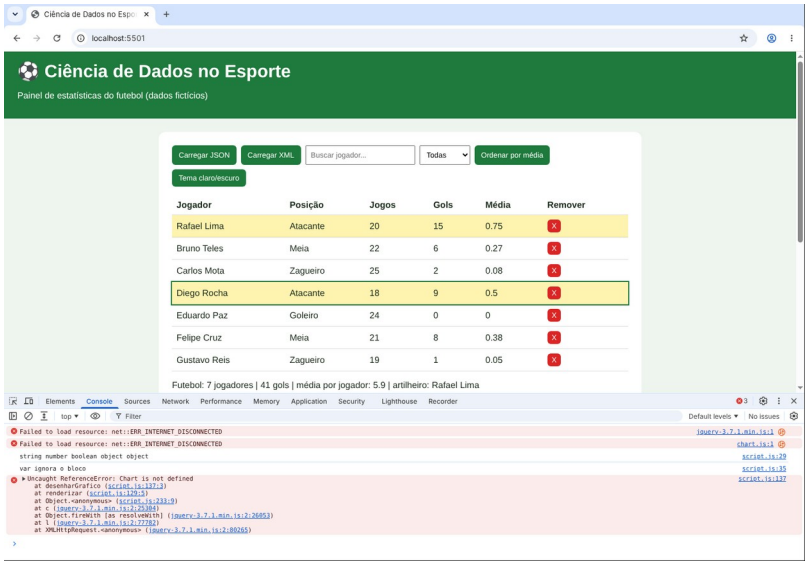
	`options: { scales: { y: { beginAtZero: true, ticks: { precision: 0 } } } }`
	`});`
	`}`
	`grafico.data.labels = lista.map(j => j.nome);`
	`grafico.data.datasets[0].data = lista.map(j => j.gols);`
	`grafico.data.datasets[0].backgroundColor = cores;`
	`grafico.update();`
	`}`

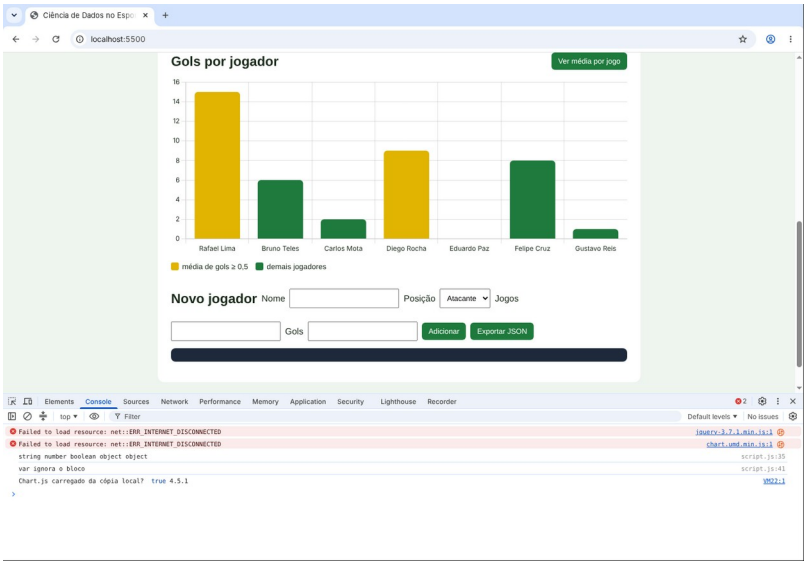
O que adaptamos e por quê	• **Tema não atualizava:** o botão de tema chama aplicarTema(), que não chama o gráfico. Então a cor das letras só mudava quando a tabela era redesenhada. Colocamos uma linha [NOVO] dentro de aplicarTema() que chama atualizarGrafico(grafico.listaAtual). • **Por que grafico foi para o BLOCO 1:** aplicarTema() roda logo no começo (linha do cookie). Se o let grafico ficasse lá no BLOCO 7, ao ler grafico antes da linha do let daria ReferenceError (variável let ainda não criada). Declarando no BLOCO 1, ela já existe com null. • **Chart.defaults.color** muda o padrão de TODOS os gráficos. Trocamos por cores em cada eixo (scales.x.ticks.color etc.), lidas das variáveis CSS --texto e --campo, assim se mudar o CSS o gráfico acompanha. • **Aviso só com .text():** trocamos por try/catch com throw, como no BLOCO 5 do projeto. Se qualquer coisa do gráfico der erro, aparece em vermelho na tela e a tabela continua funcionando. • **Extras nossos:** aviso quando a busca não deixa ninguém; botão #btnMetrica para ver média por jogo; aria-label no canvas com o resumo dos dados para leitor de tela; trocamos arrow functions por function.

Trecho final, comentado por nós	**<head> do index.html:**
	`<!-- NOVA FUNCIONALIDADE (Trilha 3) - Chart.js 4.5.1 pela CDN jsDelivr, com versão FIXA -->`
	`<!-- Versão fixa: se a biblioteca lançar uma versão nova com mudanças, nosso gráfico não quebra -->`
	`<script src="https://cdn.jsdelivr.net/npm/chart.js@4.5.1/dist/chart.umd.min.js"></script>`
	`<!-- PLANO B (sem internet): mesma ideia do jQuery. Se window.Chart não existe, usa a cópia local -->`
	`<script>window.Chart || document.write('<script src="chart.umd.min.js"><\/script>');</script>`
	
	Início do BLOCO 7 (script.js):
	`// ================================================================`
	`// BLOCO 7 [NOVO]: GRÁFICO DE GOLS POR JOGADOR (biblioteca Chart.js)`
	`// A ideia: o gráfico é criado UMA vez e depois só trocamos os dados com update().`
	`// Ele é chamado no fim do renderizar(), então se atualiza sozinho.`
	`// ================================================================`
	
	`// Lê uma variável CSS do <body> (assim o gráfico usa as mesmas cores do tema`

Item	Preenchido pela dupla

	claro/escuro)
	  function corDoTema(nomeVariavel) {
	  return getComputedStyle(document.body).getPropertyValue(nomeVariavel).trim(); // trim() tira espaços que o navegador pode devolver
	  }
	
	  function atualizarGrafico(lista) {
	  try {
	  // Se a CDN e a cópia local falharem, "Chart" não existe. Sem isso dava ReferenceError e o resto parava.
	  if (typeof Chart === "undefined") {
	  throw new Error("Gráfico indisponível: a biblioteca Chart.js não carregou (sem internet e sem o arquivo chart.umd.min.js).");
	  }
	
	  // map() cria um array NOVO a partir de outro: aqui, uma lista só com os nomes
	  const nomes = lista.map(function (j) { return j.nome; });
	  // Operador ternário escolhe o valor de cada barra: gols ou média (mediaGols já existe no BLOCO 3)
	  const valores = lista.map(function (j) { return metrica === "gols" ? j.gols : mediaGols(j); });
	  // Mesma regra da tabela: média >= LIMITE_DESTAQUE ganha a cor de destaque
	  const cores = lista.map(function (j) { return mediaGols(j) >= LIMITE_DESTAQUE ? "#e0b400" : corDoTema("--campo"); });
	  const corTexto = corDoTema("--texto"); // cor das letras (muda no tema escuro)
	  const corGrade = "rgba(136, 136, 136, 0.25)"; // linhas de fundo bem fraquinhas

A IA errou ou inventou algo?	Ela esqueceu que o tema é trocado por outra função (aplicarTema) e não pelo renderizar(), então a parte do tema não funcionava ao clicar no botão. Percebemos testando: clicamos em "Tema claro/escuro" e as letras do gráfico continuaram escuras no fundo escuro. Também usou Chart.defaults, que é global, e arrow functions fora do nosso padrão.
Prints (Console F12 visível)	 Print 3: ANTES. Versão da IA sem internet: "Chart is not defined" e

Item	Preenchido pela dupla
	a mensagem do JSON some.  Print 4: DEPOIS. Versão final sem internet: as CDNs falham (vermelho), mas o plano B carrega o Chart.js 4.5.1 local e o gráfico aparece.

4 Código final comentado

Abaixo está todo o código novo. No script.js, as linhas novas fora do BLOCO 7 estão marcadas com [NOVO]. Os arquivos completos estão no ZIP (pasta PainelEsporte_NovaFunc).

4.1 index.html (seção nova)

```
    <!-- SEÇÃO NOVA: gráfico de gols por jogador (Chart.js desenha dentro do <canvas>) -->
    <section class="grafico" aria-labelledby="tituloGrafico">
      <div class="grafico-topo">
        <h2 id="tituloGrafico">Gols por jogador</h2>
        <!-- troca o que o gráfico mostra: total de gols ou média por jogo -->
        <button id="btnMetrica" type="button" aria-pressed="false">Ver média por jogo</button>
      </div>
      <!-- a caixa com altura fixa evita que o gráfico cresça sem parar (responsive do Chart.js) -->
      <div class="grafico-caixa">
        <!-- role="img" + aria-label: o leitor de tela lê um resumo, já que o canvas é só desenho -->
        <canvas id="graficoGols" role="img" aria-label="Gráfico de gols por jogador"></canvas>
      </div>
      <!-- legenda das cores (mesma regra da tabela) e avisos de erro do gráfico -->
      <p class="grafico-legenda"><span class="cor-destaque"></span> média de gols ≥ 0,5   <span
class="cor-normal"></span> demais jogadores</p>
      <p id="graficoAviso" role="status"></p>
    </section>
```

4.2 style.css (bloco novo)

```
/* ===== NOVA FUNCIONALIDADE: GRÁFICO DE GOLS (Chart.js) ===== */
.grafico { margin-top: 24px; }
.grafico-topo { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; justify-content: space-
between; }
.grafico-topo h2 { margin: 0; }
.grafico-caixa { position: relative; height: 320px; margin-top: 12px; }  /* o Chart.js ocupa 100% desta
caixa */
.grafico-legenda { font-size: .85rem; display: flex; flex-wrap: wrap; align-items: center; gap: 6px; }
.cor-destaque, .cor-normal { display: inline-block; width: 14px; height: 14px; border-radius: 3px; }
.cor-destaque { background: #e0b400; }   /* mesma ideia do --destaque da tabela */
.cor-normal { background: var(--campo); }
#btnMetrica[aria-pressed="true"] { background: #14532d; }  /* botão "apertado" fica mais escuro */
button:focus-visible { outline: 3px solid #e0b400; outline-offset: 2px; } /* foco visível no teclado */
```

4.3 script.js (BLOCO 7 completo)

```
  // ================================================================
  // BLOCO 7 [NOVO]: GRÁFICO DE GOLS POR JOGADOR (biblioteca Chart.js)
  // A ideia: o gráfico é criado UMA vez e depois só trocamos os dados com update().
  // Ele é chamado no fim do renderizar(), então se atualiza sozinho.
  // ================================================================

  // Lê uma variável CSS do <body> (assim o gráfico usa as mesmas cores do tema claro/escuro)
  function corDoTema(nomeVariavel) {
    return getComputedStyle(document.body).getPropertyValue(nomeVariavel).trim(); // trim() tira espaços
que o navegador pode devolver
  }
```

```
  function atualizarGrafico(lista) {
    try {
      // Se a CDN e a cópia local falharem, "Chart" não existe. Sem isso dava ReferenceError e o resto
parava.
      if (typeof Chart === "undefined") {
        throw new Error("Gráfico indisponível: a biblioteca Chart.js não carregou (sem internet e sem o
arquivo chart.umd.min.js).");
      }

      // map() cria um array NOVO a partir de outro: aqui, uma lista só com os nomes
      const nomes = lista.map(function (j) { return j.nome; });
      // Operador ternário escolhe o valor de cada barra: gols ou média (mediaGols já existe no BLOCO 3)
      const valores = lista.map(function (j) { return metrica === "gols" ? j.gols : mediaGols(j); });
      // Mesma regra da tabela: média >= LIMITE_DESTAQUE ganha a cor de destaque
      const cores = lista.map(function (j) { return mediaGols(j) >= LIMITE_DESTAQUE ? "#e0b400" :
corDoTema("--campo"); });
      const corTexto = corDoTema("--texto");           // cor das letras (muda no tema escuro)
      const corGrade = "rgba(136, 136, 136, 0.25)";    // linhas de fundo bem fraquinhas

      if (grafico === null) {                          // 1ª vez: cria o gráfico
        grafico = new Chart(document.getElementById("graficoGols"), {
          type: "bar",                                 // gráfico de barras
          data: { labels: [], datasets: [{ label: "Gols", data: [], backgroundColor: [], borderRadius: 6 }]
},
          options: {
            responsive: true,                          // acompanha o tamanho da tela
            maintainAspectRatio: false,                // usa a altura da .grafico-caixa do CSS
            plugins: { legend: { display: false } },   // a legenda de cores já está no HTML
            scales: {
              x: { ticks: {}, grid: {} },
              y: { beginAtZero: true, ticks: {}, grid: {} } // eixo Y começa no zero
            }
          }
        });
      }

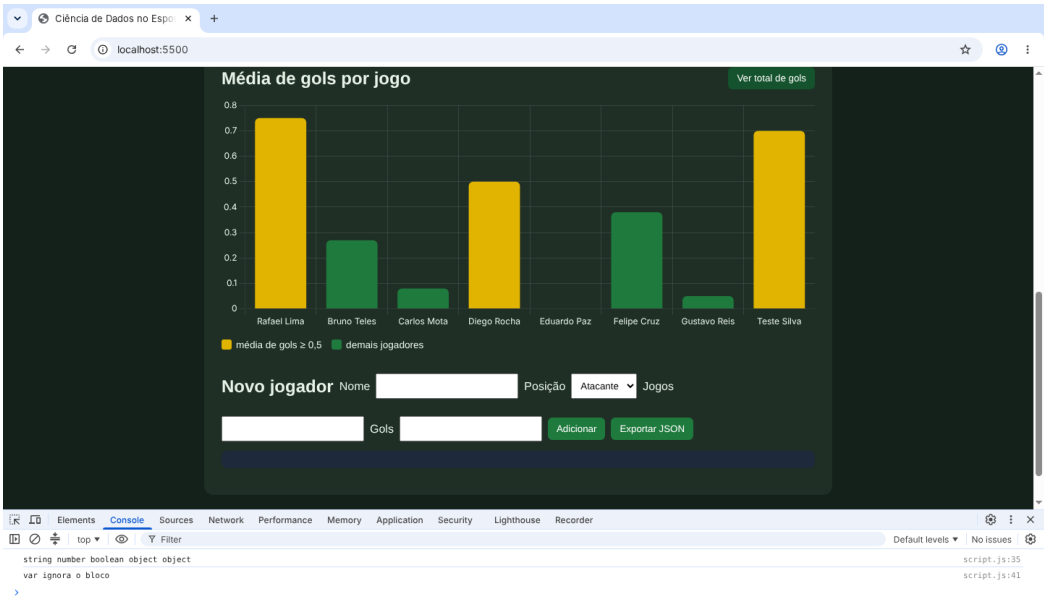
      // Daqui pra baixo roda SEMPRE: troca os dados do gráfico que já existe
      grafico.listaAtual = lista;                      // guardo a lista para o aplicarTema() redesenhar
com ela
      grafico.data.labels = nomes;
      grafico.data.datasets[0].data = valores;
      grafico.data.datasets[0].backgroundColor = cores;
      grafico.data.datasets[0].label = metrica === "gols" ? "Gols" : "Média de gols por jogo";
      // gols são inteiros (precision 0); média tem casas decimais (precision 2)
      grafico.options.scales.y.ticks.precision = metrica === "gols" ? 0 : 2;
      // cores das letras e da grade em cada eixo (forEach passa por "x" e "y")
      ["x", "y"].forEach(function (eixo) {
        grafico.options.scales[eixo].ticks.color = corTexto;
        grafico.options.scales[eixo].grid.color = corGrade;
      });
      grafico.update();                                // manda o Chart.js redesenhar com os dados novos

      // Acessibilidade: o canvas é só desenho, então escrevo um resumo no aria-label
      const resumo = lista.map(function (j, i) { return j.nome + " " + valores[i]; }).join(", ");
      $("#graficoGols").attr("aria-label", "Gráfico de " + (metrica === "gols" ? "gols" : "média de gols")
+ " por jogador: " + (resumo || "sem jogadores"));
```

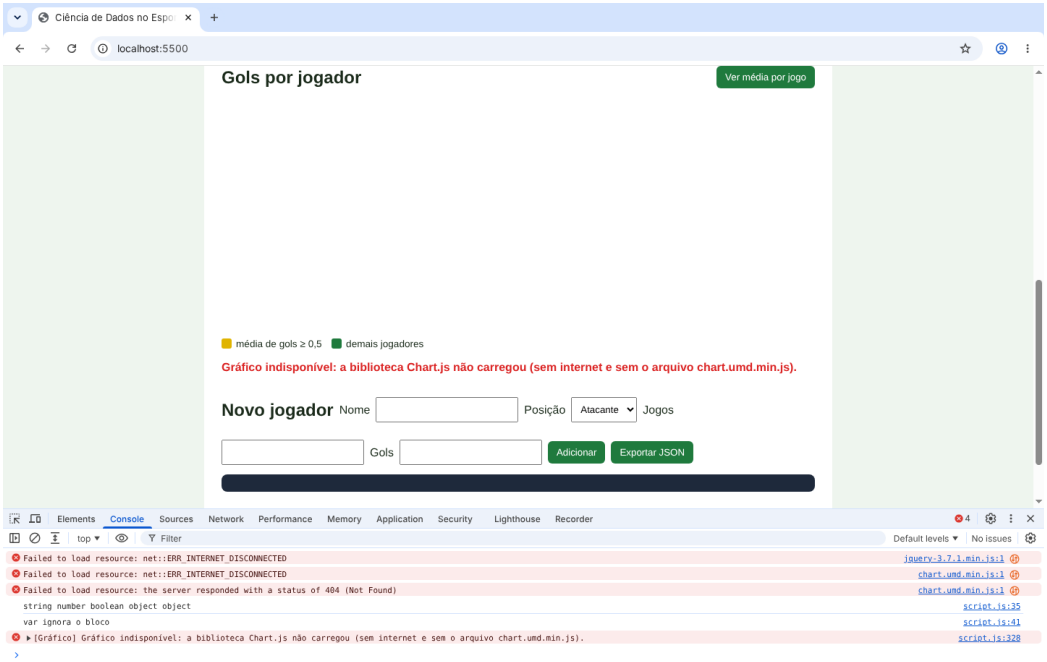
```
    // Aviso na tela quando a busca/filtro não deixa ninguém
    if (lista.length === 0) {
      $("#graficoAviso").text("Nenhum jogador para mostrar no gráfico.").attr("class", "erro");
    } else {
      $("#graficoAviso").text("").attr("class", "");
    }
  } catch (erro) {
    // Mostra o erro na TELA (não só no Console) e deixa a tabela funcionando normalmente
    $("#graficoAviso").text(erro.message).attr("class", "erro");
    console.error("[Gráfico]", erro.message);
  }
}

// Botão que alterna o gráfico entre "gols" e "média por jogo"
$("#btnMetrica").on("click", function () {
  metrica = metrica === "gols" ? "media" : "gols";   // inverte a métrica
  $(this).text(metrica === "gols" ? "Ver média por jogo" : "Ver total de gols") // texto do botão
         .attr("aria-pressed", metrica === "media");  // avisa o leitor de tela se está "apertado"
  $("#tituloGrafico").text(metrica === "gols" ? "Gols por jogador" : "Média de gols por jogo");
  renderizar();                                       // redesenha tabela + gráfico
});
```

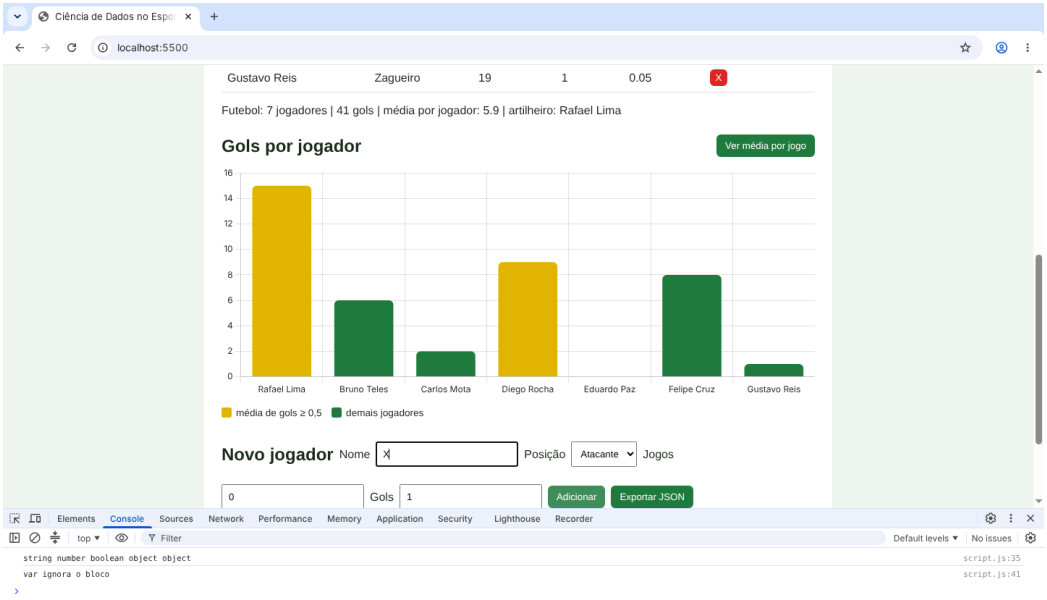
5 Funcionalidade funcionando



Print 5: adicionamos "Teste Silva" (10 jogos, 7 gols), ligamos a média por jogo e o tema escuro. O gráfico mudou sozinho e o Console está limpo.



Print 6: sem internet e sem o arquivo local do Chart.js. O aviso aparece na tela e no Console; a tabela continua funcionando.



Print 7: jogador com 0 jogos. O try/catch do projeto mostra o erro e o gráfico não muda.

6 Registro de testes

Testamos pelo servidor local na porta 5500 (mesmo endereço do Live Server) com o Console aberto. Para "sem internet" usamos F12 > Network > Offline. As cinco primeiras linhas são as pedidas no modelo do professor.

Teste que fizemos	O que deveria acontecer	O que aconteceu	OK?
Abrir a página pelo Live Server	Funcionalidade aparece sem erro no Console	7 barras; erros: 0	Sim
Clicar no botão da funcionalidade (Adicionar jogador)	Gráfico ganha uma barra nova sozinho	8 barras, msg: Jogador adicionado!	Sim
Dado inválido (jogador com 0 jogos)	Mensagem de erro na tela, gráfico igual	msg: O jogador precisa ter ao menos 1 jogo.	Sim
Arquivo ausente (jogadores.json com erro 404)	Mensagem de erro na tela (não só no Console)	Falha ao carregar o JSON. Use o Live Server.	Sim
Sem internet (F12 > Network > Offline)	Plano B carrega o Chart.js local	plano B local carregou; 7 barras	Sim
Abrir com duplo clique (file://)	AJAX bloqueado, mensagem vermelha	Falha ao carregar o JSON. Use o Live Server.	Sim
Sem internet e sem chart.umd.min.js	Aviso na tela, tabela continua	tabela 7 linhas; aviso: Gráfico indisponível: a biblioteca Chart.js não carregou (sem internet e sem o arquivo chart.umd.min.js).	Sim
Remover jogador (botão X)	Barra some do gráfico	7 barras	Sim
Filtro "Atacante"	Gráfico mostra só atacantes	Rafael Lima, Diego Rocha	Sim
Busca "zzz" (ninguém)	Aviso "Nenhum jogador..."	Nenhum jogador para mostrar no gráfico.	Sim
Ordenar por média	Gráfico na mesma ordem da tabela	Rafael Lima, Diego Rocha, Felipe Cruz	Sim
Botão Ver média por jogo	Barras mostram a média	Média de gols por jogo: 0.75, 0.5, 0.38	Sim

Teste que fizemos	O que deveria acontecer	O que aconteceu	OK?
Tema claro/escuro	Letras do gráfico trocam de cor	cor do texto #1b2b1b -> #e8f3e8	Sim
Carregar XML	Gráfico com os 7 jogadores do XML	XML carregado: 7 jogadores.	Sim
Console durante todos os testes acima	Nenhum erro vermelho	Console limpo	Sim
Tela de celular (375 px)	Sem rolagem para o lado	largura da página 375px	Sim

Testes da versão da IA (antes das adaptações): busca gerou "Canvas is already in use" (Print 1); sem internet gerou "Chart is not defined" (Print 3); o filtro de posição não mudava o gráfico; o tema não mudava as cores do gráfico.

7 Justificativa da biblioteca (Chart.js)

Pergunta	Resposta
Por que escolhemos?	Desenhar um gráfico de barras "na mão" exigiria calcular escala, eixos, posição de cada barra e redesenhar tudo no <canvas> a cada mudança. O jQuery só mexe no DOM, ele não desenha gráficos. O Chart.js faz isso com poucas linhas, já é responsivo, tem animação e tooltip, e o método update() combina com nosso renderizar(): mudamos os dados e ele redesenha.
O que sabemos sobre ela? (para que serve, como instalamos, versão)	Biblioteca JavaScript de código aberto para gráficos em <canvas> (barras, linhas, pizza etc.), mantida pela comunidade Chart.js Contributors. Versão 4.5.1, arquivo chart.umd.min.js (versão "UMD", que funciona com <script> comum, sem npm). Instalamos de dois jeitos: pela CDN jsDelivr com versão fixa e com uma cópia local na pasta do projeto, igual ao jQuery. Usamos: new Chart(canvas, {type, data, options}), chart.data, chart.options e chart.update().
Quais os riscos? (internet, licença, CDN fora do ar)	Depende de internet para a CDN; resolvemos com a cópia local (testado offline, Print 4). Se a CDN e o arquivo local falharem, mostramos aviso e a tabela continua (Print 6). Licença MIT: pode usar e distribuir, desde que mantenha o aviso de copyright, que já está no topo do chart.umd.min.js. O arquivo tem cerca de 200 KB, mais pesado que o resto do projeto. Se a versão não for fixada, uma atualização pode quebrar o código.
Como faríamos sem ela?	Um gráfico de barras simples com HTML e CSS: para cada jogador, um com width em % calculado pelos gols (gols ÷ maior número de gols × 100), criado no renderizar() com $(" ").css("width", ...). Funciona, mas sem eixos, sem escala automática e sem animação.

8 Fechamento e reflexão

8.1 O que aprendemos que não sabíamos antes de usar a IA?

Aprendemos que uma biblioteca pode "guardar estado": o Chart.js lembra do gráfico que está no canvas, por isso não dá para criar outro por cima, tem que guardar numa variável e usar update(). Também entendemos na prática por que let tem escopo de bloco e não pode ser usada antes da linha onde é declarada, porque isso quase quebrou o tema. E vimos que o "plano B" do jQuery serve para qualquer biblioteca de CDN.

8.2 Em que ponto a IA mais atrapalhou?

Quando ela respondeu sem olhar o projeto inteiro. No primeiro código ela usou o array jogadores e ignorou a busca e o filtro que já estavam no renderizar(), e na parte do tema esqueceu que o botão chama outra função. O código parecia certo e não dava erro, só dava resultado errado, então só percebemos testando cada botão e comparando o gráfico com a tabela.

8.3 Onde a IA errou (resumo)

Erro da IA	Como corrigimos
new Chart() a cada renderizar() ("Canvas is already in use")	Criar uma vez e usar update()
Gráfico ignorava busca e filtro	Passar listaGrafico, montada no próprio renderizar()
CDN sem versão e sem plano B ("Chart is not defined")	chart.js@4.5.1 + window.Chart || document.write com cópia local
Tema não atualizava o gráfico	Linha [NOVO] em aplicarTema() + variável grafico no BLOCO 1
Chart.defaults (global) e arrow functions	Cores por eixo lidas do CSS e function () {} no padrão do projeto

8.4 Roteiro para a apresentação (3 minutos)

- **0:00 a 0:30:** projeto, trilha e o que o gráfico faz (mostrar na tela).
- **0:30 a 1:15:** demonstrar: adicionar jogador, filtro, média, tema, Console limpo.
- **1:15 a 2:15:** explicar o BLOCO 7: criar uma vez (grafico === null), trocar dados, update(), try/catch.
- **2:15 a 3:00:** erros da IA (canvas e tema) e por que escolhemos o Chart.js com plano B.

9 Checklist de entrega

- ☒ Pasta do projeto em .zip, abrindo pelo Live Server (PainelEsporte_NovaFunc.zip, com README.md).
- ☒ Word de evidências com prompts, código da IA, adaptações, comentários e prints.
- ☒ Registro de testes (seção 6).
- ☒ Justificativa da biblioteca (seção 7).
- ☒ PDF deste documento.
- ☐ Ensaio da explicação oral de 3 minutos (seção 8.4).

Referências

CHART.JS. **Chart.js documentation**. Disponível em: https://www.chartjs.org/docs/latest/. Acesso em: 4 nov. 2026.

CHART.JS. **Updating Charts**. Disponível em: https://www.chartjs.org/docs/latest/developers/updates.html. Acesso em: 4 nov. 2026.

JSDELIVR. **chart.js CDN by jsDelivr**. Disponível em: https://www.jsdelivr.com/package/npm/chart.js. Acesso em: 4 nov. 2026.

JQUERY. **jQuery API Documentation**. Disponível em: https://api.jquery.com/. Acesso em: 4 nov. 2026.

MDN WEB DOCS. **Window: getComputedStyle()**. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Window/getComputedStyle. Acesso em: 4 nov. 2026.

IBM. **O que é uma rede de distribuição de conteúdo (CDN)?** Disponível em: https://www.ibm.com/br-pt/think/topics/content-delivery-networks. Acesso em: 4 nov. 2026.

SOUZA, Clayton de Almeida. **Atividade: Nova Funcionalidade com IA** e projeto base Painel do Esporte. Interface Web II, ETEC Taboão da Serra, 2026. Material de aula.